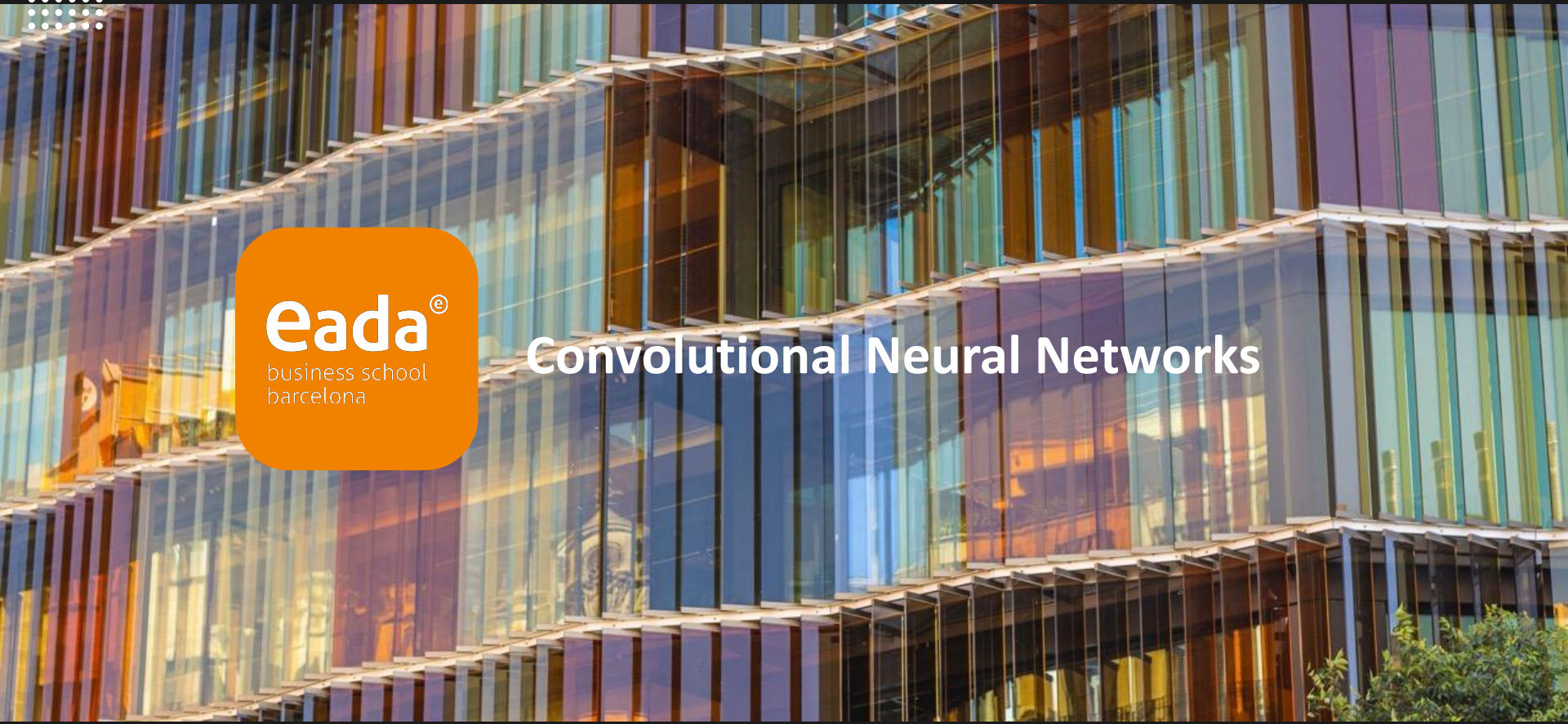


eada business school barcelona

Convolutional Neural Networks

SESSION 7 · RECAP

Where Session 6 Left Us

- MLP trained on image features.
- MLP trained on image pixels, flattened.
- What results did you obtain?
    - Did the MLP beat the previous machine learning models?
    - Are the features the ceiling?

TODAY

Today the network sees the **pixels in their original distribution**, and learns its own filters.

SESSION 7

Learning Objectives

1. **Explain** why flattening an image for an MLP wastes its structure and explodes the parameter count.
2. **Describe** a conv layer as Session 4's filters made learnable: local connectivity + weight sharing.
3. **Compute by hand** convolution, ReLU, max pooling and the output shape of every layer.
4. **Explain** feature hierarchies, skip connections (ResNet) and why transfer learning works.
5. **Fine-tune** a pretrained ResNet-18 on MILK10k and compare it with every model of the course.

SESSION 7

Today's Agenda

PART 1 — THEORY	BREAK	PART 2 — HANDS-ON PYTHON
80 min	20 min	80 min

- **Part 1:** convolutional layers, pooling, feature hierarchies, classic architectures, transfer learning.
- **Break:** 20 minutes.
- **Part 2:** build a CNN from scratch in PyTorch, then fine-tune a pretrained ResNet on MILK10k.

SESSION 7

Part 1 — Theory

Teaching a network to see structure, not just numbers

eada business school barcelona

Why Not Just Flatten the Image?

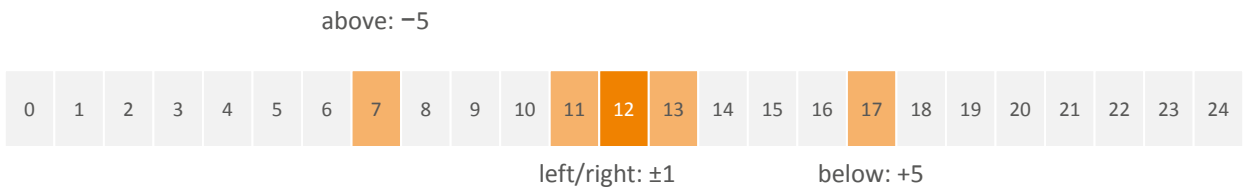
? *Nearby pixels are related. Does an MLP that receives a flattened image even know which pixels are neighbors?*

TOY IMAGE 5×5

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

pixel 12 and its 4 neighbors

FLATTENED: ONE LONG LIST OF 25 NUMBERS



IN A REAL 224×224 IMAGE

the pixel just above is **224 positions** away in the list (672 counting the 3 colors)

- An MLP has a separate weight for every position: it does not know which inputs are neighbors, and must relearn it everywhere.
- Shuffle all pixels the same way for every image: an MLP trains **exactly as well**. A useful 'image' model should not survive that.

The Parameter Explosion

? *One hidden layer of 128 neurons on a flattened 224×224 RGB image: how many weights?*

DENSE LAYER · 150,528 → 128

150,528 × 128 + 128

19,267,712

one weight per pixel, per neuron

CONV LAYER · 32 filters of 3×3

3 × 3 × 3 × 32 + 32

896

the same 27 weights reused at every position

KEY IDEA

Same input, **21,500× fewer weights**, and the conv layer's output is still an image: 32 feature maps of 224×224. Fewer weights = less to learn from a small dataset = less overfitting.

SESSION 7 · THEORY · BY HAND

Convolution by Hand: a Recap of Session 4

INPUT · toy lesion

100	100	100	100	100
100	100	100	100	100
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

$*$

KERNEL · Sobel x

-1	0	1
-2	0	2
-1	0	1

$=$

OUTPUT · 3×3

-90	-90	0
-270	-270	0
-360	-360	0

TOP-LEFT VALUE

row 1: −100 + 0 + 100 = 0
row 2: −200 + 0 + 200 = 0
row 3: −100 + 0 + 10 = **−90**
= **−90** (lesion on the right)

- Slide the 3×3 window over the image; at each position multiply the 9 pairs and add: **one output value**.
- Big |value| = the pattern the kernel describes is there (here: a vertical edge, bright→dark).
- No padding, stride 1: 5×5 in, 3×3 out. With padding 1 the output stays 5×5 ("same").

SESSION 7 · THEORY

A Conv Layer = Session 4's Filters, Made Learnable

? *In Session 4 we chose all 9 numbers of every kernel. What if gradient descent chose them?*

SESSION 4 · WE DESIGNED THE 9 NUMBERS

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

blur

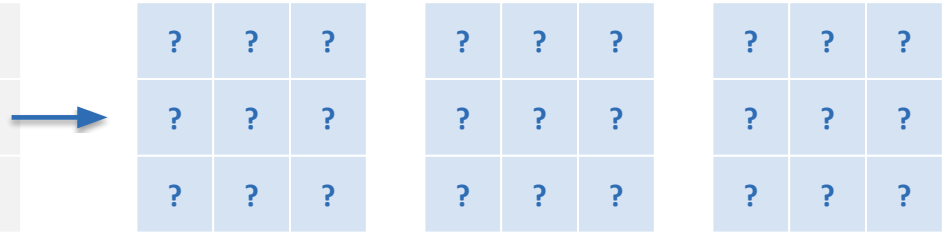
-1	0	1
-2	0	2
-1	0	1

Sobel x

0	1	0
1	-4	1
0	1	0

Laplacian

SESSION 7 · BACKPROP LEARNS THEM



KEY IDEA

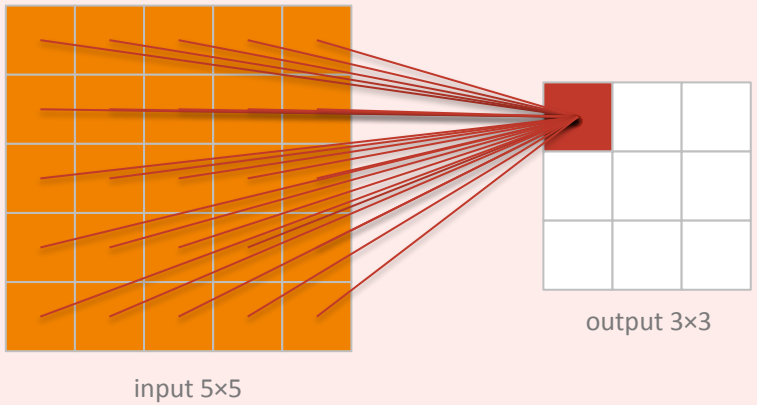
The kernel numbers are **weights**, like the w's of Session 6: they start random, and backprop gives ∂L/∂(each kernel number).

A layer learns many filters in parallel (16, 32, 64…), each free to specialise: edges, colors, hairs...

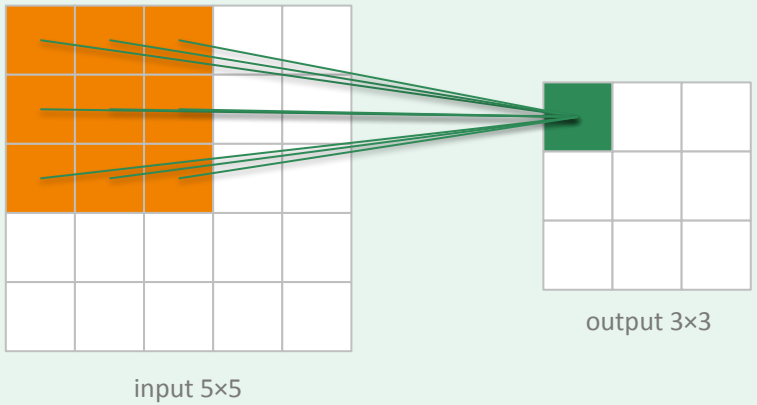
Gradient descent finds the filters that actually help the diagnosis.

Two Ideas: Local Connectivity and Weight Sharing

DENSE · each output sees all 25 inputs

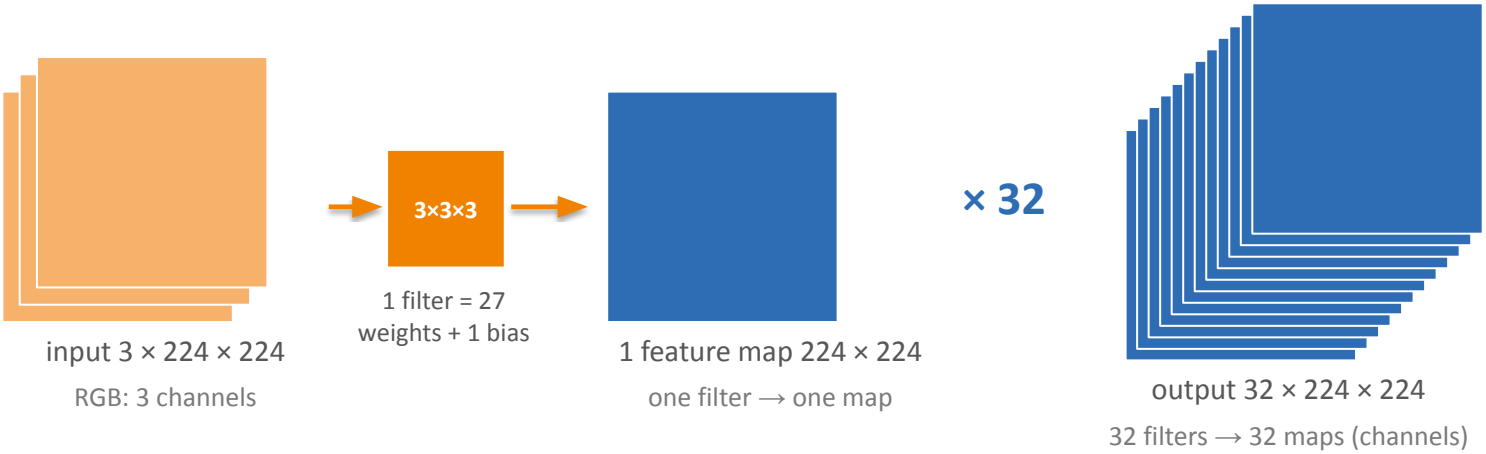


CONV · each output sees a 3×3 window



	Dense (25 → 9)	Conv 3×3 (25 → 9)
weights per output value	25	9 (one window)
different weights in total	25 × 9 = **225**	**9** (shared everywhere)

Many Filters, Many Channels

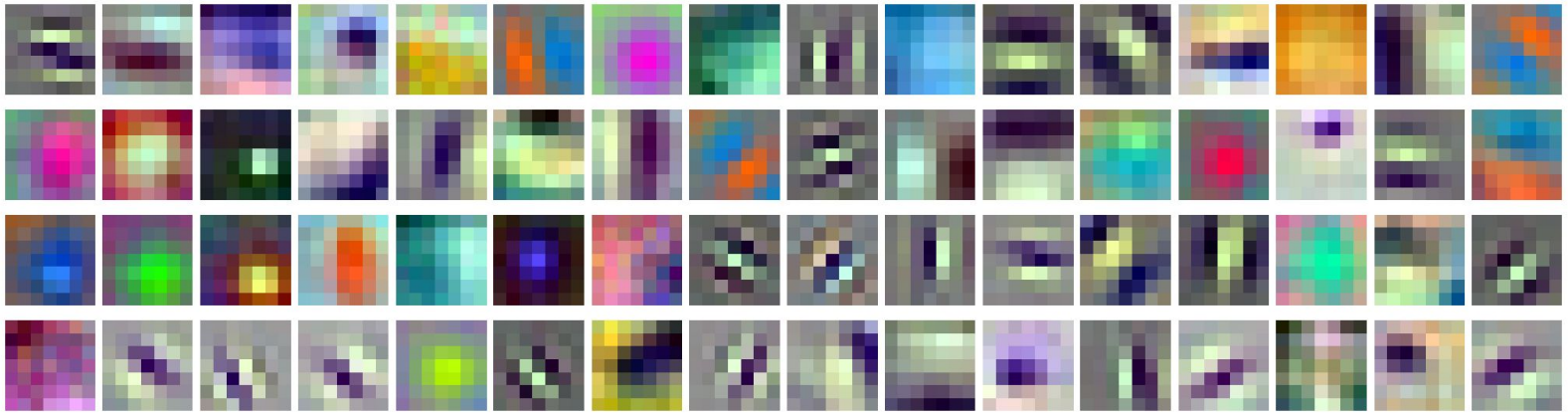


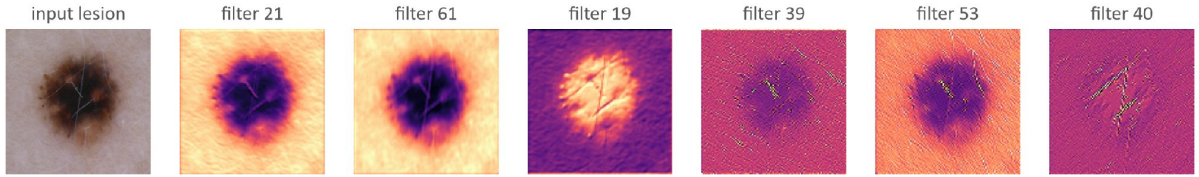

```
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, stride=1, padding=1)
```

argument	meaning	effect
in_channels = 3	input depth	must match the previous layer's channels
out_channels = 32	number of filters	= number of output feature maps
kernel_size, stride, padding	Session 4	decide the output height and width

What Learned Filters Look Like

RESNET-18, FIRST LAYER: 64 FILTERS OF 7×7×3, LEARNED ON 1.28 MILLION IMAGENET PHOTOS





Their feature maps on our lesion: 3 color/blob detectors, 3 edge detectors that fire on the **hairs**.

KEY IDEA

Nobody designed these.
Gradient descent rediscovered **Sobel and blur**.

ReLU and Max Pooling by Hand

FEATURE MAP 4×4

-3	5	2	0
1	8	-1	4
0	-2	6	7
3	1	-4	2

ReLU →

AFTER ReLU · negatives → 0

0	5	2	0
1	8	0	4
0	0	6	7
3	1	0	2

max 2×2 →

MAX POOL · 2×2

8	4
3	7

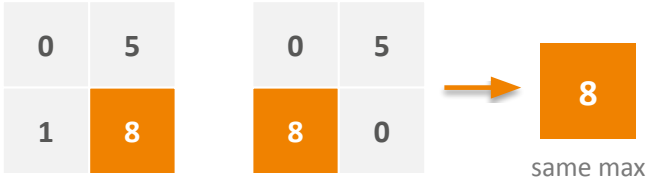
each color = one 2×2 window; keep its largest value

- **ReLU**: keep the evidence that a pattern is there, drop the negatives (Session 6).
- **Max pooling**: 'was the pattern anywhere in this window?' Keeps the strongest answer, throws away its exact position.
- 4×4 → 2×2: **4× fewer numbers** for the next layer, and **no weights** to learn.

Why Pool? Invariance and a Growing View

? *After learning good filters, why would we deliberately throw information away?*

1 · SMALL SHIFTS DON'T MATTER

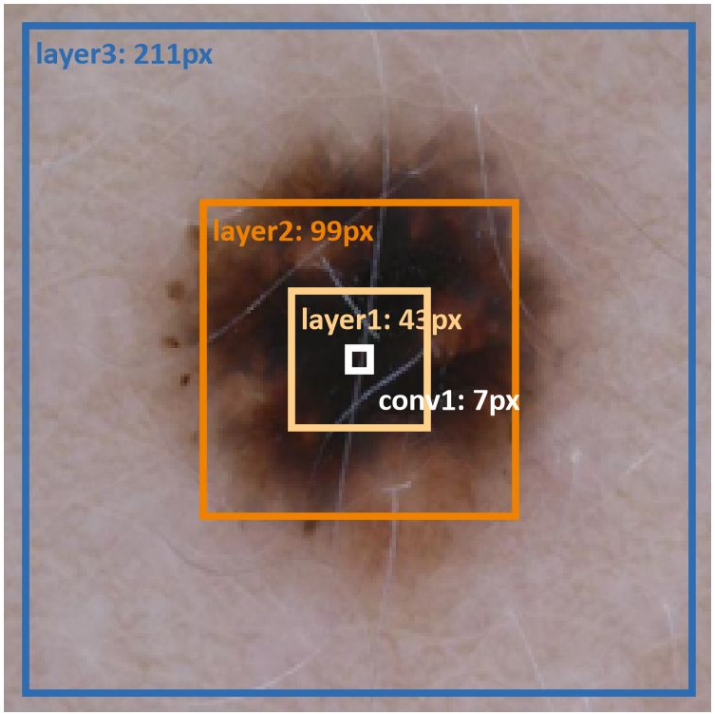


2 · FEWER NUMBERS

2×2 pooling: 224 ⟶ 112 ⟶ 56 … each pool keeps ¼ of the values.

3 · EACH LAYER SEES FURTHER

A neuron deep in the network depends on a large patch of the image: its **receptive field**.

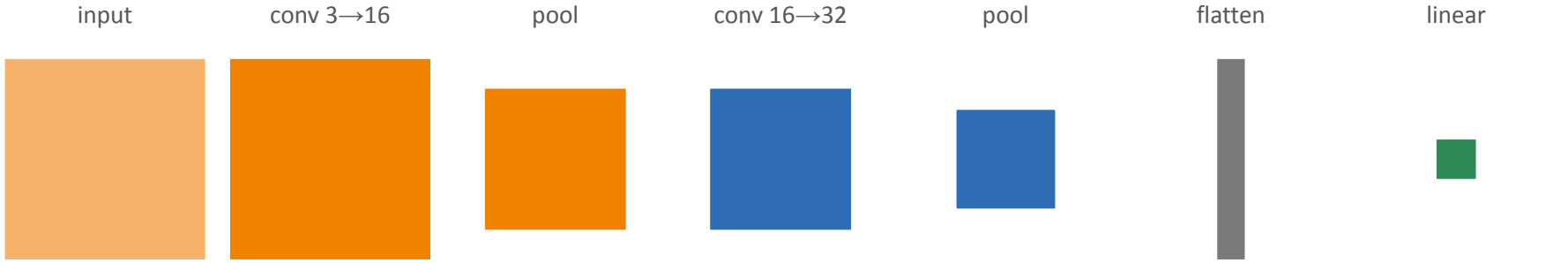


ResNet-18 layer4: 435 px, the whole image

Tracing Shapes and Parameters by Hand

$$\text{out} = \lfloor (\text{in} + 2 \cdot \text{pad} - \text{kernel}) / \text{stride} \rfloor + 1$$

conv 3×3, pad 1, stride 1: (224 + 2 − 3)/1 + 1 = 224 (same size) · max pool 2×2, stride 2: (224 − 2)/2 + 1 = 112 (half)



KEY IDEA

Total 306,147 parameters, and **98%** sit in the last Linear layer.
Modern CNNs average each map to one number instead (global average pooling): 32·3 + 3 = 99.

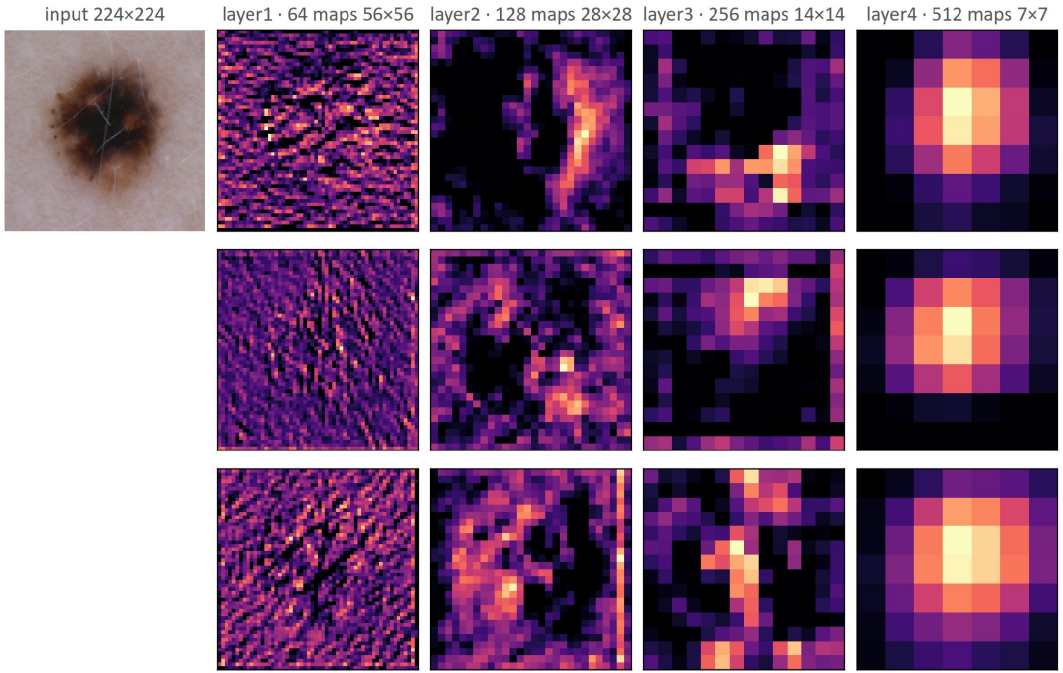
Anatomy of a CNN: Backbone and Head



- The **backbone** turns pixels into a feature vector. Today: learned. Session 5: our 33 handcrafted numbers played this role.
- The **head** is Session 6's network: from features to 3 class scores.
- Training is Session 6's loop, unchanged: forward, loss, backward, step. The kernels are just more weights.
- This split is what makes **transfer learning** possible: keep a good backbone, replace the head.

Feature Hierarchy: What Each Layer Sees

? *Should the first layer of a CNN detect edges, or whole lesions?*



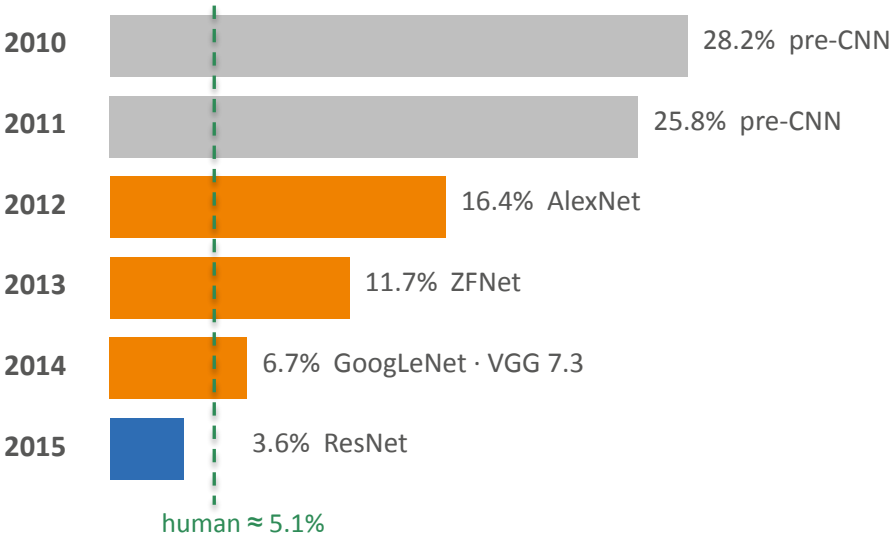
depth	typical features
early	edges, color blobs, hairs (Sobel-like)
middle	textures, dots, dermoscopic network patterns
late	lesion-level parts: border, globules, structure
last	abstract, task-specific; 7×7 maps, 512 channels

Real ResNet-18 maps on lesion ISIC_0073863: resolution goes down, channels and receptive field go up.

Classic Architectures: a Short History

? *From 2012 to 2015, what made each new winner possible?*

IMAGENET TOP-5 ERROR (%), WINNER OF EACH YEAR

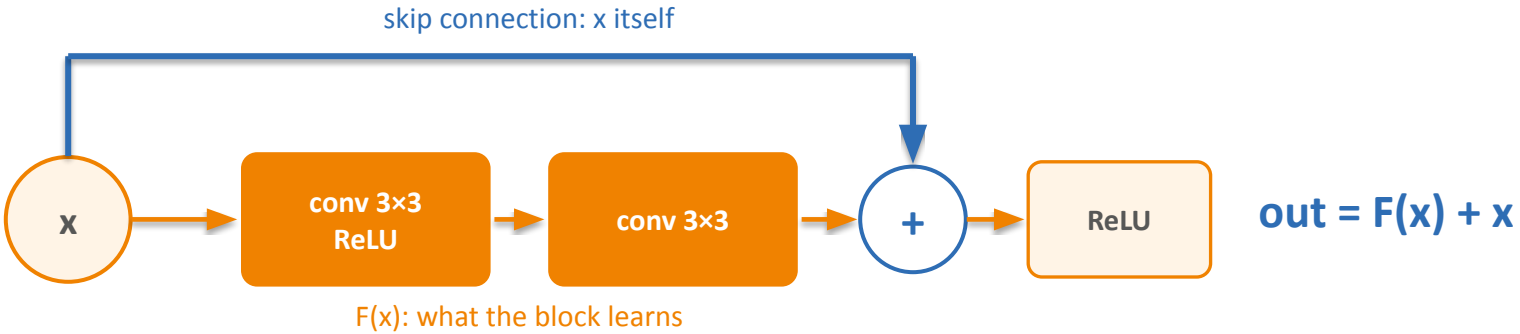


model	year	key idea
LeNet-5	1998	first practical CNN, digits, 5 layers
AlexNet	2012	8 layers, **ReLU**, dropout, trained on **GPUs**
VGG	2014	16–19 layers of only 3×3 convs
ResNet	2015	152 layers thanks to **skip connections**

The top-5 error is the percentage of images where the correct class is not among the model's 5 highest-scoring guesses, so a 3.6% top-5 error means that for about 96 images out of 100, the right answer was somewhere in its top 5, out of ImageNet's 1,000 classes

ResNet and Skip Connections

? *Adding layers should never hurt. Why did 50-layer plain networks train worse than 20-layer ones?*



- Plain deep nets: the gradient is a product of many slopes and **vanishes** (Session 6: 0.25^{10}).
- With a skip, $\partial \text{out}/\partial x = \partial F/\partial x$ **+ 1**: the gradient always has a direct highway back to the early layers.
- Worst case the block learns F(x) = 0 and just passes x on: an extra block can't make things worse. ResNet-18 = 8 such blocks.

  ResNet-18 has 8 residual blocks of 2 conv layers each, which makes 16, plus the first convolution and the final classification layer: 18 layers with weights, hence the name.

Why Not Train From Scratch?

? *MILK10k gives us 3,593 training lesions. ResNet-18 has 11.7 million weights. Should we train it from zero?*

IMAGENET

1,281,167

training images, 1,000 classes

MILK10K (OURS)

3,593

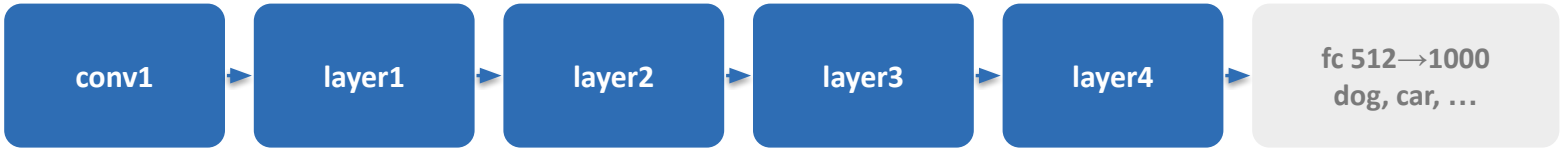
dermoscopic training lesions, 3 classes

- **357×** less data, and 11.7 M / 3,593 **≈ 3,250 weights per lesion**: a recipe for memorising (Session 6's overfitting).
- But edges, colors, blobs and textures are **generic**: a network that learned them on 1.28 M photos does not need to relearn them for skin.
- So: borrow the backbone, retrain the head. In Part 2 you'll compare both, from scratch vs pretrained, on the same lesions.

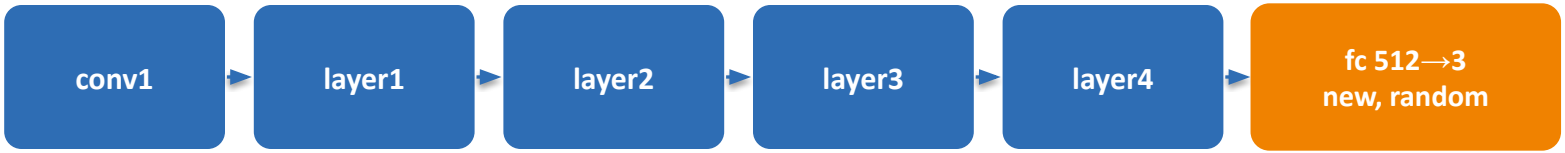
SESSION 7 · THEORY

Transfer Learning: the Idea

1 · TRAINED ON IMAGENET (by someone else, for weeks, on GPUs)



2 · KEEP THE BACKBONE, REPLACE THE HEAD



Benign
Malignant
Indeterm.

backbone: 11,176,512 pretrained weights, reused

head: 1,539 weights

IN CODE

```
model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(512, 3)    # everything before fc keeps its ImageNet weights
```

SESSION 7 · THEORY

Feature Extraction = Session 6 with Better Features

? *If the backbone is frozen, what is left to learn?*

SESSIONS 5–6



TODAY · STRATEGY 1



KEY IDEA

Freeze the backbone and it becomes a **feature extractor**: compute 512 numbers per lesion once, then it's a tabular problem, solved with Session 6's code.

SESSION 7 · THEORY

Fine-Tuning Strategies: How Much to Unfreeze?

1 · FEATURE EXTRACTION



1,539
trainable weights

very little data, or images similar to ImageNet

2 · PARTIAL FINE-TUNING



8,395,267
trainable weights

moderate data, like MILK10k: the usual best default

3 · FULL FINE-TUNING



11,178,051
trainable weights

large datasets, or images far from natural photos

frozen

trained

Freezing = `param.requires_grad = False`: autograd skips those gradients, so training is faster.

SESSION 7 · THEORY · BY HAND

Using a Pretrained Network Correctly

1 · SPEAK IMAGENET'S LANGUAGE

Resize to 224×224, scale to [0, 1], then per channel:
(x − mean) / std,
mean = 0.485, 0.456, 0.406

red = 200 ⟶ 200/255 = 0.784 ⟶ (0.784 − 0.485)/0.229 = **1.31**

2 · SMALL LEARNING RATE

The pretrained weights are already good: nudge them, don't wreck them.

new head alone: 1e-3 · fine-tuning layers: **1e-4**

3 · AUGMENT: MORE VARIETY, SAME LABELS (Session 3)

original



horizontal flip



vertical flip



both



4 · KEEP THE CLASS WEIGHTS

Same imbalance as Session 6:
Indeterminate weight **14.43**

```
nn.CrossEntropyLoss(weight=...)
```

Part 1 Recap: Key Takeaways

1. **Flattening** throws away which pixels are neighbors and needs millions of weights (19.3 M vs 896).

2. **A conv layer** is Session 4's filter, learned: local windows + one shared kernel per filter, many filters → channels.

3. **ReLU + pooling** keep strong evidence, shrink the maps and widen what each neuron sees: edges → textures → parts.

4. **ResNet's skip connections** keep gradients alive, so very deep networks train.

5. **Transfer learning**: reuse an ImageNet backbone, replace the head, unfreeze as much as your data allows.

Before Part 2

Before we come back from the break, please prepare your environment for Part 2.

Install these libraries in the Python environment you use for the course:

```
pip install torch torchvision numpy pandas matplotlib pillow scikit-learn
```

If you've followed the earlier sessions, you probably only need `torch` and `torchvision`.
Installing them can take a few minutes, so please start right at the beginning of the break.

No GPU is needed. If you're using Google Colab, all of these are normally already installed.

To check that it worked, run:

```
import torch, torchvision
print(torch.__version__, torchvision.__version__)
```

TIME FOR A BREAK

20 Minutes

Stretch, grab a coffee, and be back on time.

eada
business school
barcelona

SESSION 7

Part 2 — Hands-on Python

Live coding on your laptops or on Colab GPU or CPU.

eada
business school
barcelona

Mini Challenge: Unfreeze More and Compare

? *Is unfreezing more layers always worth the extra training cost?*

1 TRAIN

Fresh pretrained ResNet-18,
unfreeze **layer3 + layer4 + fc**,
lr = 1e-4, same epochs

2 MEASURE

trainable parameters
seconds per epoch
best validation macro-F1

3 DECIDE

Compare with layer4 only.
Worth it? Why, or why not?
Test set stays closed.

HINT

```
for name, p in model.named_parameters():
    p.requires_grad = name.startswith(("layer3", "layer4", "fc"))
trainable = [p for p in model.parameters() if p.requires_grad]    # rebuild the optimizer with these
```

SESSION 7

Homework and Coming Up Next

HOMEWORK · BEFORE SESSION 8

1. Finalise your scoreboard (Sessions 5–7, validation and test macro-F1). Bring your best model and its number.

2. Read about EfficientNet or DenseNet: one sentence on how it differs from ResNet.

3. Optional: fine-tune the whole network (strategy 3).

NEXT · SESSION 8: VISION TASKS

So far: one label per image (**classification**).

Next: **where** is the lesion (detection) and **which pixels** belong to it (segmentation): the deep-learning versions of Session 1's three tasks.

REFERENCES

- He et al., Deep Residual Learning for Image Recognition (2015): the ResNet paper.
- Stanford CS231n notes, Convolutional Neural Networks: cs231n.github.io
- PyTorch transfer-learning tutorial: pytorch.org/tutorials/beginner/transfer_learning_tutorial.html